



Nom i Cognom:	Marc Losada Soler
Enllaç al repositori Git:	https://github.com/Marcls2004/DAM-M0486-Access-dades-PR2

Objectius:

- Aprendre a serialitzar objectes JAVA

Instruccions:

- Desenvolupa el codi necessari per a cada exercici seguint les especificacions indicades. El codi ha de superar els tests JUnit proporcionats al repositori de punt de partida (mvn test).
- En cas d'haver de contestar alguna pregunta en aquest document, cal fer-ho dins del quadre indicat.

Criteris d'avaluació:

Cada exercici tindrà la mateixa puntuació. Es valorarà:

- Format correcte del codi (indentació i compliment de la guia d'estil de Java).
- Noms clars i descriptius per a mètodes i variables.
- Qualitat general del codi, amb comentaris explicatius quan sigui necessari.
- Les còpies seran penalitzades amb una puntuació de 0.

Entrega:

- Repositori Git privat, compartit amb l'usuari jpala4-ieti, que contingui el codi complet de la pràctica.
- Subdirectori "doc" dins del repositori amb el fitxer memoria.pdf.
- URL del repositori: S'ha de pujar a Moodle.
- El repositori inclou un workflow de GitHub Actions que executa els tests a cada push: en el lliurament final el semàfor ha de quedar en verd.

Recursos i materials:

- JDK de Java versió 21, Maven i Git instal·lats.
- Eina de programació: necessiteu una IDE per programar en Java (VS Code, IntelliJ, o una altra).
- Repositori amb exemples:
<https://github.com/jpala4-ieti/DAM-M0486-RA1-Repositori-Referencia-26-27> (llegeix el fitxer README.md per obtenir instruccions sobre com executar-lo).
- Altres recursos: Podeu utilitzar Google per buscar tutorials que us ajudin a resoldre els exercicis.

Punt de partida de la pràctica:

<https://github.com/jpala4-ieti/DAM-M0486-RA1-PR1.2-Practica-Punt-Partida-26-27>



Resol els exercicis proposats. El directori de treball ha de ser 'data', de la mateixa forma que es fa en el repositori d'exemples

```
String camiBase = System.getProperty("user.dir") + "/data/";
```



Exercici 0: Gestió de dades de persones amb HashMap i arxius binaris

Modifica el programa `PR120mainPersonesHashmap.java`. El `main` ja ve fet (crea el `HashMap` i crida els dos mètodes): has d'implementar els mètodes `escriurePersones` i `llegirPersones` marcats amb `// CODI PRÀCTICA`, sense canviar-ne la signatura. Tasques:

- El `HashMap<String, Integer>` amb el nom i l'edat de 5 persones (dades predefinides) ja es crea al `main` i es passa a `escriurePersones`.
- Empra `FileOutputStream` + `DataOutputStream` per guardar aquestes dades a l'arxiu `PR120persones.dat`. Un `HashMap` no té una mida fixa, així que cal definir un format: escriu primer el nombre d'entrades (`writeInt`) i després, per a cada persona, el nom (`writeUTF`) i l'edat (`writeInt`).
- Llegeix `PR120persones.dat` amb `FileInputStream` + `DataInputStream` seguint exactament el mateix ordre (nombre d'entrades i, a continuació, cada parella nom/edat) i mostra el contingut per pantalla amb el format `Nom: edat anys` (una persona per línia).
- La ruta del fitxer serà `System.getProperty("user.dir") + "/data/PR120persones.dat"`.
- Si el fitxer no existeix o es produeix un error de lectura els mètodes han de llançar una excepció `IOFitxerExcepcio` que embolcalli l'excepció original (causa) i el missatge de la qual contingui "Error en llegir les persones del fitxer" (lectura) o "Error en escriure les persones al fitxer" (escriptura); són els textos que comproven els tests. El `main` ha de gestionar l'Excepció per evitar que l'execució es finalitzi de manera abrupta.
- **Important:** obre el fitxer amb `FileInputStream/FileOutputStream` (no amb `Files.newInputStream`): el test comprova que la causa de l'excepció en llegir un fitxer inexistent és una `FileNotFoundException`.

(Mirar exemple `EscripturaDadesPrimitives.java` i `LecturaDadesPrimitives.java` per orientació)

Dades a introduir i sortida esperada

Carla: 22 anys
Bernat: 30 anys
David: 35 anys
Anna: 25 anys
Elena: 28 anys



Exercici 1: Serialització d'un HashMap

Modifica els programes `PR121mainEscriu.java` i `PR121mainLlegeix.java` per gestionar la serialització i deserialització d'un HashMap. Realitza les següents tasques:

- La classe `PR121hashmap` (paquet `com.project.objectes`) ja ve feta al repositori: implementa `Serializable` i conté un `HashMap<String, Integer>`. Revisa-la abans de continuar.
- Desenvolupa dos programes separats (el `main` de cadascun ja ve fet; has d'implementar els mètodes marcats amb `// CODI PRÀCTICA`):
 - `PR121mainEscriu.java`:
 - La instància de `PR121hashmap` i les dades (noms i edats) ja es creen al `main`; implementa `serialitzarHashMap(PR121hashmap)`.
 - Utilitza `ObjectOutputStream` per serialitzar l'objecte `PR121hashmap` i guardar-lo a un arxiu anomenat `PR121HashMapData.ser`.
 - La ruta del fitxer serà `System.getProperty("user.dir") + "/data/PR121HashMapData.ser"`.
 - Gestiona adequadament les possibles excepcions.
 - `PR121mainLlegeix.java`:
 - Implementa `deserialitzarHashMap()`: utilitza `ObjectInputStream` per llegir l'objecte serialitzat de l'arxiu `PR121HashMapData.ser` i retorna'l (cal fer un cast a `PR121hashmap`).
 - El `main` ja mostra el contingut del `HashMap` per pantalla.
 - Genera l'excepció corresponent si el fitxer no existeix o es produeix un error de lectura (captura també `ClassNotFoundException`); el missatge ha de contenir "Error en deserialitzar l'objecte HashMap" (text que comprova el test).
- Assegura't que ambdós programes utilitzin la mateixa ruta per al fitxer:
`System.getProperty("user.dir") + "/data/PR121HashMapData.ser"`
- Si hi ha problemes treballant amb els fitxers, els mètodes han de llançar una excepció `IOFitxerExcepcio` amb un missatge descriptiu. Implementa una gestió d'errors robusta per evitar que l'execució es finalitzi de manera abrupta en cas d'errors.

(Pots consultar els exemples `EscripturaObjectes.java` i `LecturaObjectes.java` per orientació sobre com implementar la serialització i deserialització)

Sortida esperada del programa de Llegir:

Carla: 22 anys
Bernat: 30 anys
Anna: 25 anys



Exercici 2: Serialització d'objectes Persona

Modifica el programa `PR122main.java` per gestionar informació de persones mitjançant serialització d'objectes. Segueix aquestes instruccions:

1. La classe `PR122persona` (paquet `com.project.objectes`) ja ve feta al repositori: implementa `Serializable` i té els atributs `nom` (`String`), `cognom` (`String`) i `edat` (`int`), constructor, getters i setters, i un `toString()` per a una correcta visualització de l'objecte. Revisa-la abans de continuar.
2. El `main` de `PR122main.java` ja ve fet: crea la llista d'objectes i crida `serialitzarPersones` i `deserialitzarPersones`, que són els mètodes que has d'implementar. El programa ha de realitzar les següents tasques:

a. Creació d'objectes. El `main` ja crea els objectes `PR122persona` següents:

Nom	Cognom	Edat
Maria	López	36
Gustavo	Ponts	63
Irene	Sales	54

b. Serialització:

- Utilitza `ObjectOutputStream` per serialitzar la llista sencera d'objectes `PR122persona` (un `ArrayList` és `Serializable`: es pot escriure amb un sol `writeObject`).
- Guarda'ls en un arxiu anomenat `PR122persones.dat`.
- La ruta del fitxer serà `System.getProperty("user.dir") + "/data/PR122persones.dat"`.

c. Deserialització i visualització:

- Utilitza `ObjectInputStream` per llegir la llista d'objectes serialitzats de l'arxiu `PR122persones.dat` (`readObject` retorna un `Object`: cal fer un cast a `List<PR122persona>`).
- Mostra la informació de cada persona per pantalla.

3. Implementa una gestió d'errors robusta:

- Gestiona adequadament les possibles excepcions durant la serialització i deserialització.
- Si el fitxer no existeix o es produeix un error de lectura ha de generar una excepció `IOFitxerExcepcio` amb el missatge adequat; si el fitxer no existeix, el missatge ha de contenir "Fitxer no trobat" (text que comprova el test). Per distingir aquest cas, captura `FileNotFoundException` abans que `IOException` (n'és una subclasse) o comprova prèviament si el fitxer existeix.

(Pots consultar els exemples `EscripturaObjectes.java` i `LecturaObjectes.java`, i per a llistes `EscripturaLlistes.java` i `LecturaLlistes.java`, per orientació sobre com implementar la serialització i deserialització). Sortida esperada:

Maria López, 36 anys

Gustavo Ponts, 63 anys

Irene Sales, 54 anys



Exercici 3: Gestió de dades de treballadors en CSV

Completa el programa `PR123mainTreballadors.java` per gestionar informació de treballadors emmagatzemada en un fitxer CSV. Segueix aquestes instruccions:

1. Examina el fitxer `PR123treballadors.csv` del directori "data" i comprova que conté les següents dades:

Id	Nom	Cognom	Departament	Salari
123	Nicolás	Rana	2	1000.00
435	Xavi	Gil	2	1800.50
876	Daniel	Ramos	6	700.30
285	Pedro	Drake	4	2500.00
224	Joan	Potter	6	1000.00

Modifica el programa principal `PR123mainTreballadors.java` (el menú i els mètodes privats `llegirFitxerCSV`/`escriureFitxerCSV` ja venen fets; has d'implementar `mostrarTreballadors` i `modificarTreballador`) perquè realitzi les següents tasques:

2. Lectura del fitxer CSV:
 - Pots utilitzar la classe `UtilsCSV` per llegir el contingut de l'arxiu `PR123treballadors.csv`.
 - La ruta del fitxer serà `System.getProperty("user.dir") + "/data/PR123treballadors.csv"`.
 - Mostra el contingut per pantalla.
3. Interacció amb l'usuari:
 - Demana a l'usuari que introdueixi un identificador de treballador.
 - Localitza el treballador amb l'identificador proporcionat. Si no existeix cap treballador amb aquest Id, `modificarTreballador` ha de llançar `IllegalArgumentException` amb un missatge descriptiu (el menú ja la captura i mostra el missatge; el test ho comprova).
 - Demana quina dada vol modificar (Nom, Cognom, Departament o Salari). Si la columna no existeix al CSV, `modificarTreballador` també ha de llançar `IllegalArgumentException`.
 - Demana el nou valor per a la dada seleccionada.
 - Guarda les modificacions a l'arxiu `PR123treballadors.csv`.
4. Implementa una gestió d'errors robusta:
 - Gestiona adequadament les possibles excepcions durant la lectura i escriptura del fitxer CSV.
 - Si el fitxer no existeix o es produeix un error de lectura/escriptura, els mètodes han de llançar una excepció `IOFitxerExcepcio` i el `main` l'ha de tractar (`UtilsCSV.escriure` propaga la `IOException` perquè la puguis embolcallar; `UtilsCSV.llegir` retorna `null` si no pot llegir).



5. Assegura't que el programa utilitza la mateixa ruta per al fitxer tant per llegir com per escriure: `System.getProperty("user.dir") + "/data/PR123treballadors.csv"`
6. Els tests comproven: que la modificació del salari de l'Id 123 queda desada al fitxer i que la resta de línies no canvien; que un Id inexistent o una columna invàlida provoquen `IllegalArgumentException`; i que un fitxer inexistent provoca `IOFitxerExcepcio`.

(Pots consultar l'exemple `GestioCSV.java` per orientació sobre com treballar amb fitxers CSV)



Exercici 4: Registre d'estudiants amb RandomAccessFile

Modifica el programa `PR124main.java` per gestionar les notes dels estudiants d'una universitat utilitzant `RandomAccessFile`. Aquest mètode permet un accés ràpid i eficient a les dades sense necessitat de carregar tot el fitxer a memòria. Podeu mirar aquest exemple com a base [RandomAccessFilesVideojocsManager.java](#) (el teniu també en el repositori).

Descripció del problema

Una universitat necessita un sistema per gestionar les notes dels seus estudiants de manera eficient (Accés directe a dades per número de registre). Cada estudiant té un número de registre únic (en format enter) i una nota final associada.

Requisits del programa

1. Estructura del fitxer:
 - Utilitza `RandomAccessFile` per crear i gestionar un fitxer anomenat `PR124estudiants.dat`.
 - Cada registre d'estudiant ha de tenir una longitud fixa de 48 bytes amb la següent estructura:
 - Número de registre: 4 bytes (enter)
 - Nom: 40 bytes reservats, codificat en UTF-8. **El límit és de bytes, no de caràcters:** un caràcter UTF-8 ocupa entre 1 i 4 bytes, de manera que hi caben 40 caràcters ASCII però només 13 caràcters xinesos. Si el nom ocupa més de 40 bytes s'ha de truncar sense partir cap caràcter (`UTF8Utils.truncar`, com a l'exemple); si n'ocupa menys, s'omple amb bytes a zero fins als 40.
 - Nota: 4 bytes (float)
2. Funcionalitats del programa: Modificar el `PR124main.java` que permeti a l'usuari realitzar les següents accions:
 - Llistar els estudiants amb el seu número de registre.
 - Afegir un nou estudiant amb la seva nota.
 - Consultar la nota d'un estudiant mitjançant el seu número de registre.
 - Actualitzar la nota d'un estudiant existent mitjançant el seu número de registre.
3. Eficiència:
 - Utilitza els mètodes apropiats de `RandomAccessFile` (`seek`, `getFilePointer`, `length`, `skipBytes`) per posicionar el punter de lectura/escriptura directament a la posició correcta del fitxer. Llegeix els 40 bytes del nom amb `readFully` (no amb `read`, que pot llegir-ne menys).
4. Gestió d'errors:
 - Implementa una gestió d'errors robusta per tractar situacions com:
 - Intentar accedir a un estudiant que no existeix, també quan el fitxer encara no s'ha creat: `consultarNotaFitxer` i `actualitzarNotaFitxer` han de mostrar el



missatge "No s'ha trobat l'estudiant..." sense llançar cap excepció, i `l·listarEstudiantsFitxer` ha de mostrar "No hi ha estudiants registrats."

- Entrades de dades incorrectes per part de l'usuari.
 - Mostra missatges d'error apropiats (Veure secció missatges informatius més avall) sense que el programa finalitzi de manera abrupta.
5. Validació de dades:
- Assegura't que les dades introduïdes estiguin en el format correcte:
 - El número de registre ha de ser un enter positiu.
 - El nom no excedeixi el màxim fixat: aquesta validació correspon a la capa interactiva (`demanarNom`). El mètode `afegirEstudiantFitxer` **mai no ha de rebutjar un nom llarg: l'ha de truncar** a 40 bytes (els tests hi passen noms més llargs i esperen que quedin truncats).
 - La nota ha de ser un valor float entre 0 i 10.



- El número de registre no es pot repetir: si ja existeix un estudiant amb aquest registre, `afegirEstudiantFitxer` no ha d'afegir res i ha de mostrar el missatge corresponent.

Missatges informatius que ha de mostrar per pantalla

Cal que el programa informi adequadament del resultat de certes operacions.

```
// Missatge quan no hi ha estudiants registrats (fitxer buit o inexistent):
System.out.println("No hi ha estudiants registrats.");

// Missatge quan un estudiant s'ha afegit correctament:
System.out.println("Estudiant afegit correctament.");

// Missatge quan es consulta un estudiant existent:
System.out.println("Registre: " + registre + ", Nom: " + nom + ", Nota: " + nota);

// Missatge quan es vol actualitzar una nota existent:
System.out.println("Nota actualitzada correctament.");

// Missatge quan no es troba un estudiant quan es consulta per registre
System.out.println("No s'ha trobat l'estudiant amb registre: " + registre);
// Missatge quan es vol afegir un estudiant amb un número de registre que ja existeix
System.out.println("Ja existeix un estudiant amb registre: " + registre);
```

Consideracions addicionals

- Utilitza la següent ruta per al fitxer: `System.getProperty("user.dir") + "/data/PR124estudiants.dat"`
- Documenta adequadament el codi, especialment les parts relacionades amb la manipulació del `RandomAccessFile`.